

### ShopEase E-Commerce System: Object-Oriented Architecture Design

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>1. Introduction .....</b>                       | <b>1</b>  |
| <b>2. Layered Architecture Overview.....</b>       | <b>2</b>  |
| <b>2.1 Presentation Layer .....</b>                | <b>2</b>  |
| <b>2.2 Business Logic Layer.....</b>               | <b>3</b>  |
| <b>2.3 Design Patterns in Business Logic .....</b> | <b>5</b>  |
| <b>2.4 Data Access Layer .....</b>                 | <b>6</b>  |
| <b>2.4.1 Caching and File Storage .....</b>        | <b>6</b>  |
| <b>References .....</b>                            | <b>13</b> |
| <b>Use of Digital Tools .....</b>                  | <b>14</b> |

#### 1. Introduction

Modern use of e-commerce platforms requires systems that could support the handling of large product catalogues, high concurrency volume, and secure transaction workflows, this report presents the design of ShopEase, an online shopping platform architecture developed using object-oriented principles within a layered architecture.

The system is designed to achieve and ensure scalability, modularity, security, and extensibility using a three-layer architecture consisting of Presentation, Business Logic, and Data Access layers. This modular separation of architecture layers supports maintainability and system evolution over time.

## Task 'Object-Oriented Software Architecture an E-Commerce Platform' Unit 9

(Chow, 2024).

The use of Object-oriented principles, such as encapsulation and polymorphism are applied to structure system behavior in a modular way. These principles are commonly used in enterprise systems to reduce coupling and improve code maintainability.

(Freeman and Robson, 2020).

### 2. Layered Architecture Overview

#### 2.1 Presentation Layer

The Presentation Layer manages user interaction, input validation, and session handling and an `ShopEaseApp` interface acts as a façade between the user and backend services.

Input validation is being applied at this layer in order to ensure that only structurally correct data is passed into the system, such as validating email formats and preventing invalid product quantities. However, validation alone does not guarantee security and must be reinforced at backend levels.

(OWASP Foundation, 2026).

Session management uses cryptographically secure authentication tokens to support user authentication. In a stateless deployment model, signed tokens, for instance JWTs , that can reduce reliance on server-side session storage while allowing authentication information to be validated by backend services. This approach can improve scalability in distributed environments.

(Chow, 2024).

### **2.2 Business Logic Layer**

The Business Logic Layer encapsulates core system operations and enforces business rules. It is divided into three main modules.

#### **User Management Module**

The User Management Module manages user registration, authentication, and session lifecycle management. Additionally, passwords are hashed using Argon2id with integrated salt, which is a recommended approach for securing password secrecy.

(OWASP Foundation, 2026).

Data hiding is being done using encapsulation which conceal implementation details and therefore reduces the risk of sensitive operations exposure, password verification, remaining hidden within the `UserManager` class, reducing exposure of internal logic.

#### **Product Catalogue Module**

The module `ProductCatalogue` is responsible for tasks, for instance, product listing, search functionality, inventory tracking, and stock updates. Concurrency risks may arise during simultaneous purchase operations. To mitigate this, thread synchronization techniques such as reentrant locks could be utilized in prototype implementations in order to reduce the risk of race conditions that are affecting stock consistency. However, in case of distributed systems, database-level concurrency control would be typically required.

(OWASP Foundation, 2026)

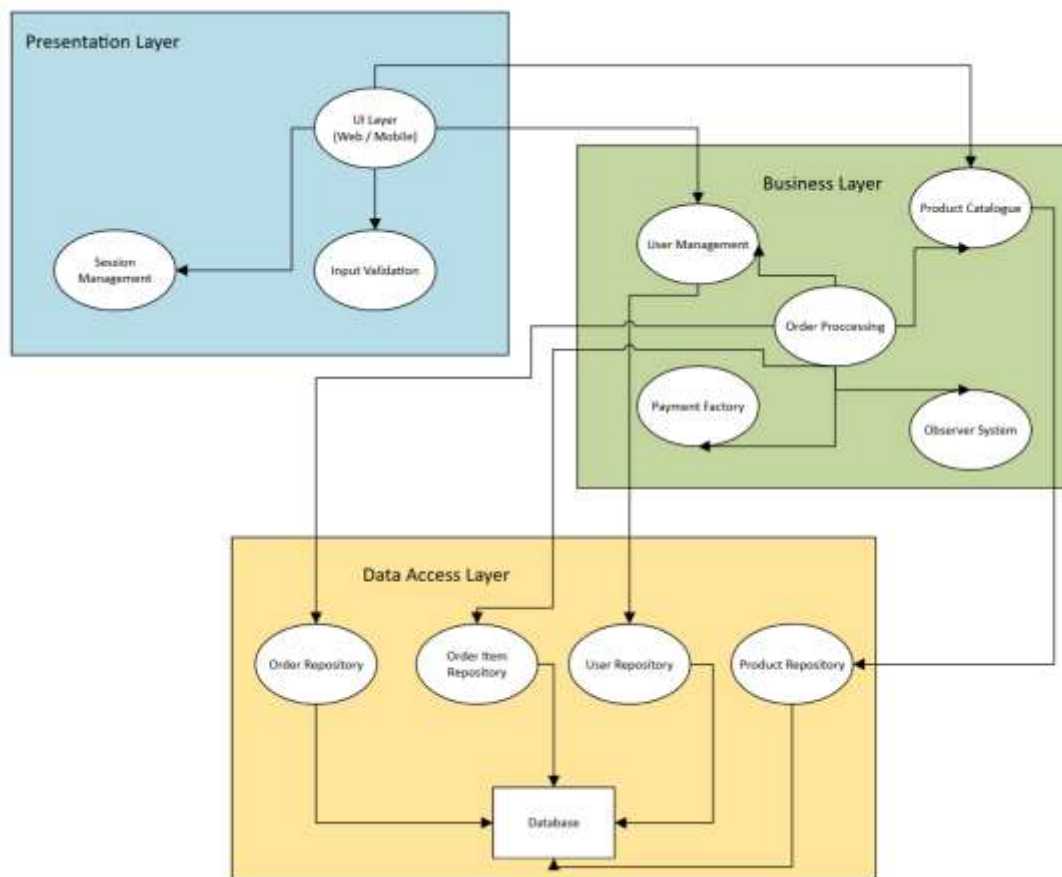
#### **Order Processing Module**

## Task 'Object-Oriented Software Architecture an E-Commerce Platform' Unit 9

The Module Order Processing is designed to control order creation, payment handling, inventory updates, and transaction logging, such method aims to preserve transactional consistency by maintaining a strict order related operations execution sequence, however, full ACID assurances would depend on underlying database transaction support rather than application-level logic alone.

(Chow, 2024).

**Figure 1: High Level Architecture**



### **2.3 Design Patterns in Business Logic**

Object-oriented design patterns are incorporated into the Business Logic Layer to sustain extensibility and loose coupling and to maintain a more flexible design behavior.

#### **Factory Method Pattern**

The Factory Method Pattern is utilized for the creation of different payment processors such as credit card and wallet-based processors, this allows new payment methods to be submitted without changing existing business logic, supporting the Open/Closed Principle.

(Freeman and Robson, 2020).

#### **Strategy Pattern**

The Strategy Pattern is used to encapsulate payment behaviors, allowing interchangeable payment methods at runtime, this is particularly useful for supporting multiple payment providers.

#### **Observer Pattern**

The Observer Pattern implements an event-driven notification mechanism, for instance, `NotificationObserver` sends user notifications and `AuditLogObserver` records system events, this help reduces coupling between order processing and auxiliary services such as logging and notifications.

#### **Dependency Injection**

## Task 'Object-Oriented Software Architecture an E-Commerce Platform' Unit 9

The dependencies are provided externally rather than instantiated within classes what improves modularity and testability by enabling the replacement of components using mock implementations during testing.

(Simion, et al, 2023).

### 2.4 Data Access Layer

The Data Access Layer abstracts persistence using the Repository Pattern.

Each major entity has a corresponding repository:

- UserRepository
- ProductRepository
- OrderRepository
- OrderItemRepository
- TransactionRepository

These repositories provide CRUD operations while hiding database implementation details. This allows migration between storage systems in this case, in-memory storage to PostgreSQL, without changing business logic.

Encapsulation is designed to support database operations isolation from business logic, improving maintainability.

(Chow, 2024).

#### 2.4.1 Caching and File Storage

In addition to the repository-based database access, the system also includes optional caching and file storage mechanisms designed to increase performance and improve flexibility, an in-memory caching layer can temporarily store frequently accessed

## Task 'Object-Oriented Software Architecture an E-Commerce Platform' Unit 9

product and user data, reducing repeated load on database from queries and improving responsibility and file storage is employed for collection and reservation of non-structured data, for instance, logs, exports, or system backups, thus enabling persistence beyond the relational database and supports recovery, compliance and monitoring activities in case needed. These resources are being accessed through the Data Access Layer or supporting repository components, ensuring they remain decoupled from business logic and consistent with the layered architecture.

### 3. Database Design

The system uses a relational model consisting of:

User (user\_id, email, password\_hash)

Product (product\_id, name, price, stock\_quantity)

Order (order\_id, user\_id, timestamp, total\_amount)

Order Item (order\_item\_id, order\_id, product\_id, quantity)

Transaction (transaction\_id, order\_id, payment\_type, status)

| Initials | Meaning     |
|----------|-------------|
| PK       | Primary Key |
| FK       | Foreign Key |

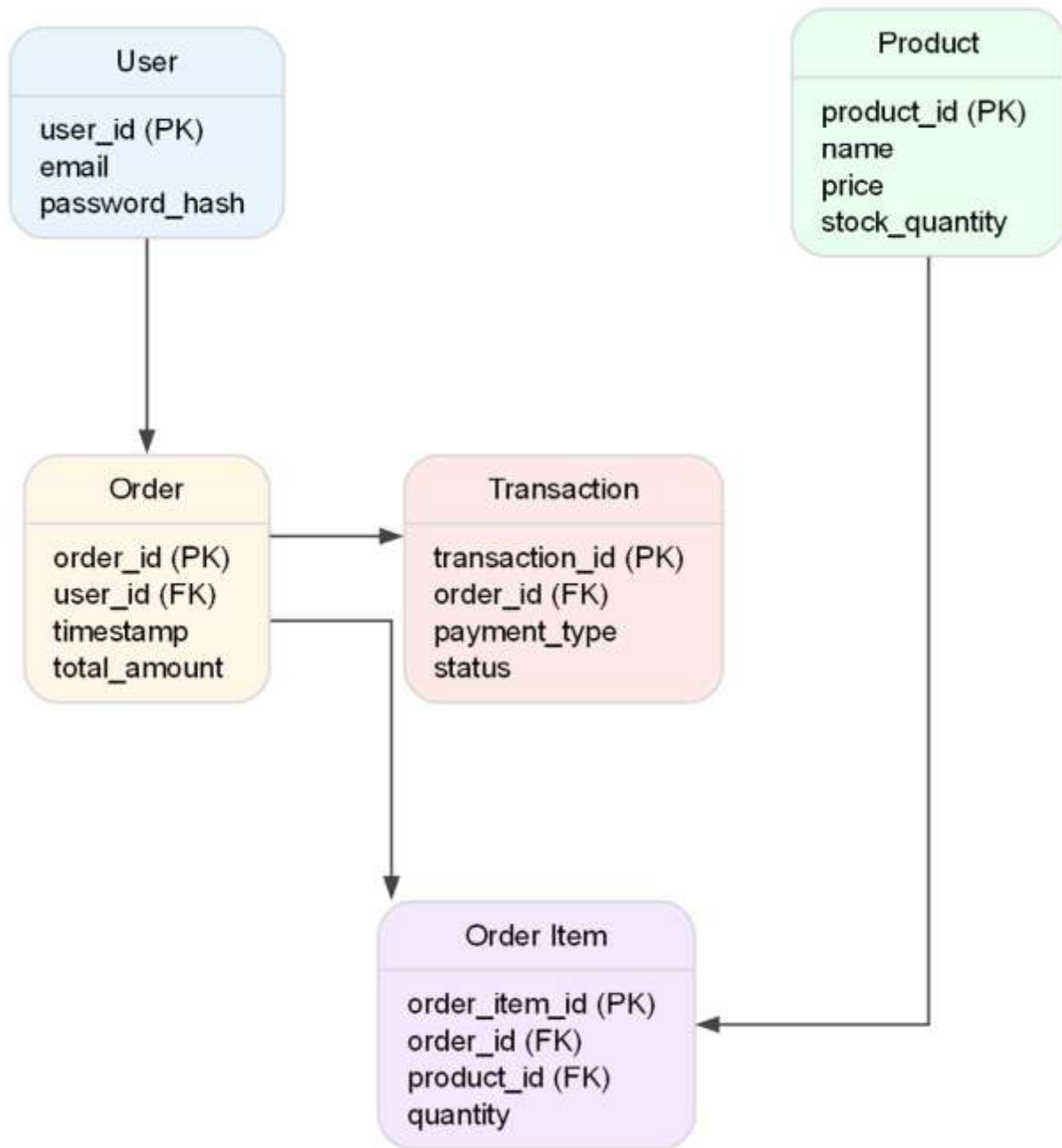
Table 1: Initials and Expression

#### Relationships

- A User can place multiple Orders (1:N)
- An Order contains multiple Products via Order Items (1:N)
- Each Order is linked to one Transaction (1:1)

This relational structure supports normalization and reduces data redundancy.

Figure 2: Database Design



#### 4. Security Implementation

Security is implemented using layered controls.

##### Authentication



## **Task ‘Object-Oriented Software Architecture an E-Commerce Platform’ Unit 9**

Passwords are being hashed using Argon2id, salt is embedded inside the generated hash string and an optional server-side pepper value to strengthen credential protection to enhance credential security and reduce the risk from a successful password cracking attack. This widely accepted approach to mitigate designed to resist GPU-accelerated brute-force such as rainbow table attacks by increasing the computational cost and preventing the reuse of precomputed hashes.

(OWASP Foundation, 2026).

PBKDF2 and bcrypt still remain widely supported in many legacy systems, Argon2id was selected in this case due to its stronger resistance to modern hardware-accelerated password cracking techniques.

### **Session Security**

Session tokens are generated using cryptographically secure random functions to reduce predictability and session hijacking risks.

### **Input Validation**

Input validation and sanitization are utilized, both on the presentation and business layers, helping to reduce invalid or potentially malicious input being executed, in addition to output encoding techniques that can be also applied in web-based interfaces to help reduce injection-related vulnerabilities. These controls complement backend validation and secure coding practices.

(OWASP Foundation, 2026).

### **Audit Logging**

## **Task ‘Object-Oriented Software Architecture an E-Commerce Platform’ Unit 9**

The Observer Pattern is used to record system events for monitoring and traceability purposes.

### **5. Scalability and Extensibility**

#### **Scalability**

The layered architecture supports independent scaling of components:

- Presentation Layer → load balancing
- Business Logic Layer → horizontal scaling
- Data Layer → database replication for reading scalability

Stateless business components allow deployment across distributed systems.

#### **Extensibility**

The system supports future enhancements through:

- Open/Closed Principle adherence
- Pluggable payment strategies
- Observer-based extensions such as recommendation systems

These principles ensure that new features can be introduced with minimal modification to existing code.

(Chow, 2024).

### **6. Concurrency and Thread Safety**

## **Task ‘Object-Oriented Software Architecture an E-Commerce Platform’ Unit 9**

In this prototype-level design thread protection is being managed using reentrant locks, this use of such, helps reducing the likelihood of a race conditions occurrence when multiple users attempt to simultaneously modify a shared resources, for instance, product stock, however, this method is limited to single-process systems, in distributed architecture, concurrency control would typically be handled through distributed locking mechanisms, or directly at the database level. Thus, the system presents foundational concurrency principles rather than a production-grade distributed concurrency management.

(McDonald, 2022).

### **7. Testing and Validation**

The architecture supports the following multiple testing strategies.

- **Unit Testing**

Individual modules are tested using mock repositories to isolate business logic.

- **Integration Testing**

Integration tests validate interactions between services and repositories.

- **Regression Testing**

Performing regression tests, helps address a degree of risk posed from modifications or refactoring activities that could unintentionally alter existing system behavior. This supports maintainability and safe extension of system functionality.

- **Concurrency Testing**

## Task 'Object-Oriented Software Architecture an E-Commerce Platform' Unit 9

Concurrency testing simulates multiple users placing orders simultaneously to evaluate system consistency.

- **Security Testing**

Different levels of security testing and code review practices can be applied to identify common vulnerabilities and maintain secure coding standards.

- **Testability Support**

Dependency Injection improves testability by decoupling components and allowing controlled substitution of dependencies during testing,

Such testing strategy supports modular structure, maintainability and safer refactoring of system components.

(Simion, et al, 2023).

### 8. Limitations and Future Work

While the system was designed based on solid object-oriented and layered design architecture principles, some degree of limitations exists due to the conceptual production-oriented architecture scope, concurrency control is limited to in-process mechanisms and does not extend to multi-instance or distributed execution environments. Likewise, transactional consistency is primarily managed through database-level guarantees rather than application-level enforcement.

Forthcoming improvements, could include, for instance, asynchronous processing for ordering workflows, implementation of external caching solutions in order to enhance performance under load and the adoption of more structured authentication frameworks such as token-based or role-based access control systems, such

## **Task ‘Object-Oriented Software Architecture an E-Commerce Platform’ Unit 9**

enhancements could support improved scalability and robustness while remaining consistent with the current layered architecture.

### **9. Conclusion**

The developed ShopEase system demonstrates the application of object-oriented design within a layered architecture and design patterns such as; Factory, Strategy, Observer, and Repository to contribute improved modularity, separation of concerns and enhanced maintainability across the landscape in addition to security mechanisms that are designed to support an improved foundational protection for user data using the implementation of hashing, input validation and session management. Scalability considerations were addressed quite early conceptually through layered separation of services which are planned without reliance on persistent in-memory state between requests thus supporting potential system growth, although implemented as a conceptual prototype, the architecture provides a structured foundation for further development into a more complete system.

### **References**

- Chow, J. (Tsz S. (2024) Software Architecture with Kotlin: Combine Various Architectural Styles to Create Sustainable and Scalable Software Solutions. 1st edition. [Online]. Birmingham: Packt Publishing, Limited. Available at: [https://essex.primo.exlibrisgroup.com/permalink/44UOES\\_INST/s7iv0e/cdi\\_proquest\\_ebookcentral\\_EBC31813189](https://essex.primo.exlibrisgroup.com/permalink/44UOES_INST/s7iv0e/cdi_proquest_ebookcentral_EBC31813189) [Accessed: 26 May 2026]

## Task 'Object-Oriented Software Architecture an E-Commerce Platform' Unit 9

- McDonald, J. C. (2022) Dead Simple Python: Idiomatic Python for the Impatient Programmer. 1st edition. New York: No Starch Press. Available at: [https://essex.primo.exlibrisgroup.com/permalink/44UOES\\_INST/s7iv0e/cdi\\_pr oquest\\_ebookcentral\\_EBC7129684](https://essex.primo.exlibrisgroup.com/permalink/44UOES_INST/s7iv0e/cdi_pr oquest_ebookcentral_EBC7129684) [Accessed: 22 May 2026]
- Freeman, E. & Robson, E. (2020) Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software. Second edition. O'Reilly. Available at: [https://essex.primo.exlibrisgroup.com/permalink/44UOES\\_INST/s7iv0e/cdi\\_as kewsholts\\_vlebooks\\_9781492077978](https://essex.primo.exlibrisgroup.com/permalink/44UOES_INST/s7iv0e/cdi_as kewsholts_vlebooks_9781492077978) [Accessed: 26 May 2026].
- OWASP Foundation (2026) OWASP Cheat Sheet Series. Available at: <https://cheatsheetseries.owasp.org/index.html> [Accessed: 24 May 2026].
- Simion, A. (2023) Test-Driven Development in Go: A practical guide to writing idiomatic and efficient Go tests through real-world examples. 1st edition. [Online]. Birmingham: Packt Publishing. Available at: [https://essex.primo.exlibrisgroup.com/permalink/44UOES\\_INST/s7iv0e/cdi\\_ie ee\\_books\\_10251201](https://essex.primo.exlibrisgroup.com/permalink/44UOES_INST/s7iv0e/cdi_ie ee_books_10251201) [Accessed: 26 May 2026].

### Use of Digital Tools

This document has been produced exclusively for educational purposes. Referenced content, copyrights, and trademarks, remain the sole property of the respective owner. ChatGPT 5.5 was used for some proofreading and clarity as well as literature exploration, language refinement, coding and programming techniques research. All

## **Task ‘Object-Oriented Software Architecture an E-Commerce Platform’ Unit 9**

academic writing, analysis, argumentation, and conclusions represent the original work of the author. AI tools were used for exploration, same way as academic literature exploration. Any use of content mentioned in this document is properly referenced to its original author and was used responsibly to ensure integrity and originality. I have used Artificial Intelligence responsibly and ethically, in accordance with the assignment brief. All AI use has been declared and evidenced.